

Project Title: Whack-a-Mole

Project Overview

This capstone project tasks you with the development of an advanced "Whack-a-Mole" game application in Java. This project transcends a simple reflex game; it is an exercise in engineering an event-driven, concurrent application that demonstrates a robust understanding and practical application of core Object-Oriented Programming (OOP) principles.

The game will feature a grid of "holes" from which various objects will randomly appear. The player's objective is to interact with these objects (e.g., click "moles") while avoiding others (e.g., "bombs") to maximize their score within a fixed time limit. The project requires the meticulous management of game state, the handling of concurrent object appearances ("spawning"), the processing of user input via a graphical user interface (GUI), and the persistence of high scores between game sessions.

Core Learning Objectives

This project is designed to synthesize and assess your mastery of the following key concepts from the course:

- **Object-Oriented Programming (OOP):** Design and implement a robust, flexible, and extensible class hierarchy for all game objects, demonstrating correct application of Abstraction, Encapsulation, Inheritance, and Polymorphism.
- **Concurrency (Multithreading):** Implement a stable, concurrent "game engine" that manages game logic on a separate thread, independent of the User Interface (UI) thread, to ensure a responsive application.
- **GUI and Event Handling (JavaFX or Swing):** Construct a responsive, event-driven graphical user interface. This includes building the game board, handling mouse events, and ensuring all UI updates are *thread-safe*.
- **Exception Handling (Custom & Built-in):** Implement comprehensive and professional error handling. This includes gracefully managing thread interruptions (InterruptedException), handling file I/O exceptions, and creating custom exception types for game-specific and data-related error conditions.
- **Data Persistence (File I/O & Serialization):** Design and implement a high-score system that correctly serializes and deserializes score data to and from a persistent file.

Module 1: The Core OOP Hierarchy (Abstraction & Polymorphism)

Concept: The foundation of this project is a design that is flexible and extensible. A simple game could be implemented with if statements, but this project requires a design that can accommodate new game objects (e.g., ExtraLifeMole, SlowDownTimeMole) without-rewriting the core game logic. This is achieved through abstraction and polymorphism.

Requirements:

1. **Abstract Class:** You must design and implement an abstract class named HoleOccupant.
 - This class will serve as the base for all objects that can appear from a "hole."
 - An abstract class is required (rather than an interface) because all occupants will share common *state* (e.g., a boolean flag for visibility, an integer for time remaining) and potentially common *implemented behavior* (e.g., a non-abstract hide() method or a tick() method to count down its lifespan).
2. **Abstract Contract:** The HoleOccupant class must define the abstract "contract" for all subclasses. At a minimum, it must include:
 - public abstract int whack();
 - This method is invoked when the player successfully "whacks" (clicks) the object. It must return an integer representing the score change (e.g., positive for a mole, negative for a bomb).
 - public abstract Image getImage(); (or ImageIcon for Swing)
 - This method returns the visual representation of the object to be displayed on the GUI.
3. **Concrete Subclasses:** You must implement *at least three* concrete subclasses that extend HoleOccupant:
 - Mole: The standard target. Its whack() method returns a positive score (e.g., +100).
 - Bomb: A penalty object. Its whack() method returns a large negative score (e.g., -500) or perhaps triggers a game-over condition.
 - BonusMole: A special, high-value target. Its whack() method returns a large positive score (e.g., +1000) or provides a non-score bonus (e.g., adds 5 seconds to the game clock).
4. **Polymorphism Mandate:** The main game logic (e.g., in a GameGrid or GameBoard class) *must* maintain a collection of the *abstract* type (e.g., HoleOccupant or List<HoleOccupant>).
 - When a user clicks a hole, your event handler *must not* use instanceof or type-checking to determine what was clicked.
 - Your code must simply invoke the polymorphic method (e.g., int scoreChange = gameGrid.getOccupant(holeID).whack();). The specific behavior of whack() will be determined at runtime by the concrete object, demonstrating polymorphism.

Module 2: The Game Engine (Concurrency & State Management)

Concept: A common flaw in game development is to run the game logic on the same thread as the UI. This freezes the application, as the UI cannot respond to user input while the game loop is "sleeping." This project requires you to separate the game logic into its own thread.

Requirements:

1. **Runnable Implementation:** You must create a `GameEngine` class that implements `Runnable`. This class will encapsulate the entire game loop and manage the game's state (score, time remaining, object positions).
2. **Thread Creation:** Your main application class (e.g., the class that sets up the GUI) must create and start *one* instance of this `GameEngine` on a new Thread. (e.g., `Thread gameThread = new Thread(new GameEngine(this)); gameThread.start();`).
3. **Game Loop Logic:** The `run()` method of your `GameEngine` is the heart of the project. It must contain a loop (e.g., `while (gameIsRunning)`) that executes for the duration of the game. Inside this loop, the engine will:
 - Decrement the game timer and check for game-over conditions.
 - Manage the lifecycle of `HoleOccupant` objects (e.g., call a `tick()` method on all active occupants, making them hide after their time expires).
 - Randomly select a hole and "spawn" a new `HoleOccupant` (e.g., change an `EmptyHole` object to a `Mole` or `Bomb` object).
4. **Pacing:** The game loop must pause for a set interval to control the game's pace (e.g., `Thread.sleep(1000);`). This call is required and will force you to handle the `InterruptedException`, which is a central part of Module 5.

Module 3: The GUI and Event Handling (JavaFX / Swing)

Concept: This module focuses on creating the user-facing portion of the game and, most importantly, ensuring it can communicate *safely* with the concurrent `GameEngine` thread from Module 2.

Requirements:

1. **GUI Framework:** You must implement the GUI using *either* JavaFX or Java Swing. JavaFX is the modern standard and is recommended, but Swing is also acceptable.
2. **Game Board Layout:** The main game window must display the score, the time remaining, and the grid of "holes." The grid *must* be implemented using an appropriate layout manager:
 - **JavaFX:** Use a `GridPane`. Each "hole" can be an `ImageView` or a `Button` placed within a cell of the `GridPane`.
 - **Swing:** Use a `JPanel` with a `GridLayout`. Each "hole" can be a `JLabel` (with an `ImageIcon`) or a `JButton`.
3. **Event Handling:** You must attach mouse click event listeners to each "hole" in the grid.

- **JavaFX:** Use the `node.setOnMouseClicked(event -> {... });` lambda expression.
 - **Swing:** Use the `button.addActionListener(event -> {... });` lambda expression.
 - When a hole is clicked, this handler will trigger the logic that calls the `whack()` method (from Module 1) on the `HoleOccupant` at that position.
4. **CRITICAL REQUIREMENT (Thread Safety):** The `GameEngine` thread (from Module 2) *is not permitted* to directly update any GUI components. Attempting to change an `ImageView`'s image or a `JLabel`'s icon from a background thread will cause an `IllegalStateException` or other concurrency-related exceptions.
- All UI updates that originate from the `GameEngine` (e.g., "show a mole in this hole," "update the score label") *must* be marshaled back to the dedicated UI thread using the correct mechanism.
 - **JavaFX:** You must use `Platform.runLater(new Runnable() {... });`.
 - **Swing:** You must use `SwingUtilities.invokeLater(new Runnable() {... });`.
 - Failure to adhere to this "single-thread rule" for UI updates will result in an unstable application and a failing grade for this module.

Module 4: Data Persistence (File I/O & Serialization)

Concept: To make the game complete, the high-score list must be persistent. This module requires you to save and load game data using Java's object serialization mechanism.

Requirements:

1. **Serializable Data Class:** You must create a plain old Java object (POJO) class named `PlayerScore`. This class *must* implement `java.io.Serializable`. It should contain, at a minimum, the player's name (`String`) and their final score (`int`).
2. **High Score Manager:** You must create a `HighScoreManager` class. This class is responsible for all file I/O operations and will encapsulate the persistence logic. It must provide, at minimum, two methods:
 - `public void saveScores(List<PlayerScore> scores)`
 - `public List<PlayerScore> loadScores()`
3. **Serialization Mandate:** You *must* use `ObjectOutputStream` and `ObjectInputStream` to write and read your high-score data.
 - Your `saveScores` method will serialize and write an entire `ArrayList<PlayerScore>` object to a file (e.g., "scores.dat").
 - Your `loadScores` method will deserialize and read the `ArrayList<PlayerScore>` object from the same file.
4. **Error Handling:** Your methods must correctly handle `IOException` and `ClassNotFoundException`. This provides the foundation for Module 5.

Module 5: Robust Exception Handling (Custom Exceptions)

Concept: Professional-grade code does not just "crash" when something goes wrong; it anticipates and gracefully handles errors. This module requires you to implement a robust, multi-layered exception handling strategy.

Requirements:

1. **InterruptedException Handling:** The `Thread.sleep()` call in your `GameEngine` (from Module 2) *must* be placed within a try-catch block for `InterruptedException`.
 - An empty catch block is *forbidden*. This exception is a signal, not an error.
 - You must implement a graceful shutdown mechanism. When the main application window is closed, your window-closing event handler *must* call the `interrupt()` method on your `gameThread` instance. This will *cause* the `InterruptedException` to be thrown. Your catch block should then use this signal to cleanly exit the `run()` method's loop (e.g., set `gameIsRunning = false;`) and allow the thread to terminate.
2. **Custom Checked Exception:** You must create a custom *checked* exception class named `HighScoreException` that extends `Exception`.
 - The `loadScores()` and `saveScores()` methods in your `HighScoreManager` (from Module 4) must be declared to throw `HighScoreException`.
 - If an `IOException` or `ClassNotFoundException` is caught during the deserialization process, it *must* be caught, wrapped in your new `HighScoreException`, and re-thrown (e.g., `throw new HighScoreException("Failed to load scores", e);`).
 - Your main application code must try to call `loadScores()` on startup, catch this `HighScoreException`, and handle it gracefully (e.g., by displaying an alert to the user and starting with an empty high-score list).
3. **Custom Unchecked Exception:** You must create a custom *unchecked* exception class named `InvalidGameStateException` that extends `RuntimeException`.
 - This exception should be thrown to indicate an "impossible" or unrecoverable *programmer error* in your game's state logic (e.g., if code attempts to spawn an object in a hole that is already occupied). This demonstrates that you understand the difference between recoverable (checked) and unrecoverable (unchecked) exceptions.

Submission Deliverables

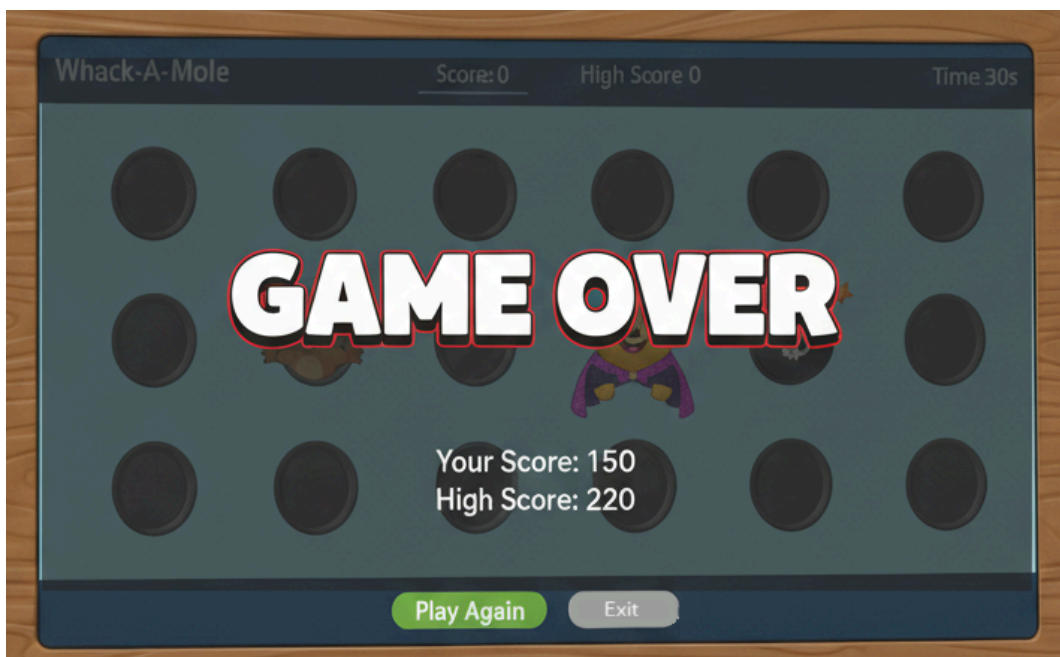
1. **Complete Java Source Code:** All .java files, fully commented and organized into appropriate packages.
2. **UML Class Diagram:** A digital diagram illustrating the complete class structure of your project. It must clearly show the inheritance hierarchy (e.g., `HoleOccupant` and its subclasses) and composition relationships (e.g., `Game` *has a* `GameEngine` and a `GameGrid`).
3. **Technical Documentation (1-2 pages):** A brief report that explains:
 - Your final OOP design and a justification for your key design choices.
 - A detailed explanation of your concurrency strategy, describing how the `GameEngine` thread and the GUI thread interact safely.
 - A description of your custom exceptions and the rationale for their use.

Visual Mockup

Visual and Functional Guidelines: The provided mockups offer a strong visual representation of the desired outcome. While successful implementation of all core functionalities and technical specifications remains the primary evaluation criterion, students are encouraged to design their user interface as close to these visual examples as practically possible. Exact visual replication is not mandatory, but a strong effort towards this aesthetic is appreciated.



Normal Game Screen



Game Over Screen